2024127662 임광윤

3. KNN

- Use dataset “PimaIndiansDiabetes2” (in R package “mlbench”) to answer questions below.

Q1: Load “PimaIndiansDiabetes2” dataset and check if there are missing values in it. If there are missing values, remove them.

Q2: Create train, valid and test set in 3:1:1 ratio. The dependent variable is “diabetes” . Create the independent variable dataset and dependent variable dataset for each train, valid, and test set, respectively.

Q3: Conduct a kNN classification. When k is 3, predict the dependent variable “diabetes” in valid set using train set and calculate the accuracy.

Q4: Use train set and valid set to find a optimal k. Plot the results and show the optimal k and the accuracy corresponding to it.

Q5: Use the optimal k to conduct kNN classification again. This time, predict the dependent variable “diabetes” in test set using train set, and compare the results with the real values. Finally, calculate the accuracy.

KNN 분석

Q1. 데이터 로드 및 결측치 처리

Step 1: “PimaIndiansDiabetes2” 데이터셋을 로드하고 결측치가 있는지 확인하라.

Step 2: 결측치가 있다면, 제거하라.

Q2. 데이터 분할 (Train/Validation/Test)

Step 1: 데이터를 3:1:1 비율로 학습(train), 검증(valid), 테스트(test) 세트로 나누어라.

Step 2: 종속 변수 "diabetes " 를 기준으로, 각 train, valid, test 세트에 대해 독립 변수 데이터셋과 종속 변수 데이터셋을 각각 생성하라.

Q3. kNN 분류 (k=3)

Step 1: kNN 분류를 수행하세요.

Step 2: K가 3일 때, 학습 세트를 사용하여 검증 세트의 종속 변수 “diabetes”를 예측하고, 정확도를 계산하라.

Q4. 최적의 k 찾기

Step 1: 학습 세트와 검증 세트를 사용하여 최적의 k 값을 찾아라.

Step 2: 결과를 시각화하고 최적의 k 값과 이에 해당하는 정확도를 표시하라.

Q5. Test 데이터에 대해 kNN 분류

Step 1: 최적의 k 값을 사용하여 kNN 분류를 다시 수행하라.

Step 2: 이번에는 학습 세트를 사용하여 테스트 세트의 종속 변수 "diabetes"를 예측하고, 실제 값과 비교하여라. 마지막으로 정확도를 계산하라.

Q1. 데이터 로드 및 결측치 처리

```r
# 필요한 패키지 설치 및 로드
if (!requireNamespace("mlbench", quietly = TRUE)) {
  install.packages("mlbench")
}
library(mlbench)

# Q1: 데이터 로드 및 결측치 처리
data("PimaIndiansDiabetes2")
str(PimaIndiansDiabetes2)  # 데이터 구조 확인
colSums(is.na(PimaIndiansDiabetes2))  # 결측치 확인

# 결측치 제거
cleaned_data <- na.omit(PimaIndiansDiabetes2)
str(cleaned_data)  # 결측치 제거 후 데이터 구조 확인
```

```
> # Q1: 데이터 로드 및 결측치 처리
> data("PimaIndiansDiabetes2")
> str(PimaIndiansDiabetes2)  # 데이터 구조 확인
'data.frame':   768 obs. of  9 variables:
 $ pregnant: num  6 1 8 1 0 5 3 10 2 8 ...
 $ glucose : num  148 85 183 89 137 116 78 115 197 125 ...
 $ pressure: num  72 66 64 66 40 74 50 NA 70 96 ...
 $ triceps : num  35 29 NA 23 35 NA 32 NA 45 NA ...
 $ insulin : num  NA NA NA 94 168 NA 88 NA 543 NA ...
 $ mass    : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 NA ...
 $ pedigree: num  0.627 0.351 0.672 0.167 2.288 ...
 $ age     : num  50 31 32 21 33 30 26 29 53 54 ...
 $ diabetes: Factor w/ 2 levels "neg","pos": 2 1 2 1 2 1 2 1 2 2 ...
> colSums(is.na(PimaIndiansDiabetes2))  # 결측치 확인
pregnant  glucose pressure  triceps  insulin     mass pedigree      age diabetes
       0        5       35      227      374       11        0        0        0
>
> # 결측치 제거
> cleaned_data <- na.omit(PimaIndiansDiabetes2)
> str(cleaned_data)  # 결측치 제거 후 데이터 구조 확인
'data.frame':   392 obs. of  9 variables:
 $ pregnant: num  1 0 3 2 1 5 0 1 1 3 ...
 $ glucose : num  89 137 78 197 189 166 118 103 115 126 ...
 $ pressure: num  66 40 50 70 60 72 84 30 70 88 ...
 $ triceps : num  23 35 32 45 23 19 47 38 30 41 ...
 $ insulin : num  94 168 88 543 846 175 230 83 96 235 ...
 $ mass    : num  28.1 43.1 31 30.5 30.1 25.8 45.8 43.3 34.6 39.3 ...
 $ pedigree: num  0.167 2.288 0.248 0.158 0.398 ...
 $ age     : num  21 33 26 53 59 51 31 33 32 27 ...
 $ diabetes: Factor w/ 2 levels "neg","pos": 1 2 2 2 2 2 2 1 2 1 ...
 - attr(*, "na.action")= 'omit' Named int [1:376] 1 2 3 6 8 10 11 12 13 16 ...
  ..- attr(*, "names")= chr [1:376] "1" "2" "3" "6" ...
```

① PimaIndiansDiabetes2 데이터 셋에는 768개의 관측치와 9개의 변수가 존재함

② 결측치는 triceps와 insulin에서 가장 많이 나타남

③ 결측치를 모두 제거한 cleaned_data 데이터셋은

392개의 관측치와 9개의 변수가 남음

Q2. 데이터 분할 (Train/Validation/Test)

```r
# Q2: 데이터 분할 (Train/Validation/Test)
set.seed(123)  # 재현성을 위한 난수 고정
n <- nrow(cleaned_data)
train_size <- round(0.6 * n)  # 60% 학습 데이터
valid_size <- round(0.2 * n)  # 20% 검증 데이터
test_size <- n - train_size - valid_size  # 나머지 20% 테스트 데이터

indices <- sample(1:n)
train_indices <- indices[1:train_size]
valid_indices <- indices[(train_size + 1):(train_size + valid_size)]
test_indices <- indices[(train_size + valid_size + 1):n]

train_data <- cleaned_data[train_indices, ]
valid_data <- cleaned_data[valid_indices, ]
test_data <- cleaned_data[test_indices, ]

# 독립 변수와 종속 변수 분리
train_x <- train_data[, -9]
train_y <- train_data[, 9]
valid_x <- valid_data[, -9]
valid_y <- valid_data[, 9]
test_x <- test_data[, -9]
test_y <- test_data[, 9]
```

Data	
cleaned_data	392 obs. of 9 variables
PimaIndiansDiabetes2	768 obs. of 9 variables
test_data	79 obs. of 9 variables
test_x	79 obs. of 8 variables
train_data	235 obs. of 9 variables
train_x	235 obs. of 8 variables
valid_data	78 obs. of 9 variables
valid_x	78 obs. of 8 variables
Values	
indices	int [1:392] 179 14 195 306 118 299 229 244 391 374 ...
n	392L
test_indices	int [1:79] 19 245 189 297 103 122 300 322 124 247 ...
test_size	79
test_y	Factor w/ 2 levels "neg","pos": 1 1 2 1 2 1 1 1 1 1 ...
train_indices	int [1:235] 179 14 195 306 118 299 229 244 391 374 ...
train_size	235
train_y	Factor w/ 2 levels "neg","pos": 2 1 1 1 2 1 2 1 1 1 ...
valid_indices	int [1:78] 333 175 327 104 283 218 176 156 2 130 ...
valid_size	78
valid_y	Factor w/ 2 levels "neg","pos": 2 1 1 2 1 2 1 2 2 1 ...

① 데이터를 학습(train), 검증(validation), 테스트(test) 세트로 3:1:1 비율로 분할

② 학습 세트: 60% (235개) / 검증 세트: 20% (78개) / 테스트 세트: 20% (79개)

③ 데이터는 독립 변수와 종속 변수 (diabetes)로 각각 분리

독립 변수: train_x / valid_x / test_x

종속 변수: train_y / valid_y / test_y

Q3. kNN 분류 (k=3)

```r
# Q3: kNN 분류 (k=3)
if (!requireNamespace("FNN", quietly = TRUE
  install.packages("FNN")
}
library(FNN)

# k=3 분류 수행
knn_pred <- knn(
  train = as.matrix(train_x),
  test = as.matrix(valid_x),
  cl = train_y,
  k = 3
)

# 정확도 계산
accuracy_q3 <- sum(knn_pred == valid_y) / l
cat("Q3: k=3일 때 검증 세트의 정확도:", acc

# 예측 값과 실제 값 비교
comparison_q3 <- data.frame(
  Actual = valid_y,
  Predicted = knn_pred
)
print(comparison_q3)  # 실제 값과 예측값 확인
```

```
> # Q3: kNN 분류 (k=3)
> if (!requireNamespace("FNN", quietly = TRUE)) {
+   install.packages("FNN")
+ }
> library(FNN)
>
> # k=3 분류 수행
> knn_pred <- knn(
+   train = as.matrix(train_x),
+   test = as.matrix(valid_x),
+   cl = train_y,
+   k = 3
+ )
>
> # 정확도 계산
> accuracy_q3 <- sum(knn_pred == valid_y) / length(valid_y)
> cat("Q3: k=3일 때 검증 세트의 정확도:", accuracy_q3, "\n")
Q3: k=3일 때 검증 세트의 정확도: 0.6153846
>
```

	Actual	Predicted
1	pos	neg
2	neg	neg
3	neg	neg
4	pos	pos
5	neg	pos
6	pos	pos
7	neg	neg
8	pos	pos
9	pos	pos
10	neg	neg
11	neg	neg
12	pos	pos
13	neg	neg
14	neg	pos
15	pos	neg
16	neg	neg
17	neg	neg
18	neg	neg
19	pos	neg
20	neg	neg
21	neg	neg
22	neg	neg
23	pos	pos
24	neg	neg
25	pos	pos
26	neg	pos
27	neg	neg
28	pos	neg
29	neg	neg
30	pos	neg
31	neg	pos
32	neg	neg
33	pos	pos
34	neg	neg
35	neg	pos
36	pos	neg
37	neg	neg
38	neg	neg
39	pos	pos

① 검증 세트를 대상으로 k=3으로 설정하여 kNN 분류를 수행

② K가 3일 때, 학습 세트를 사용하여 검증 세트의 종속 변수 “diabetes”를 예측

③ **결과:** 검증 세트의 정확도: **61.54%**

Q4. 최적의 k 찾기

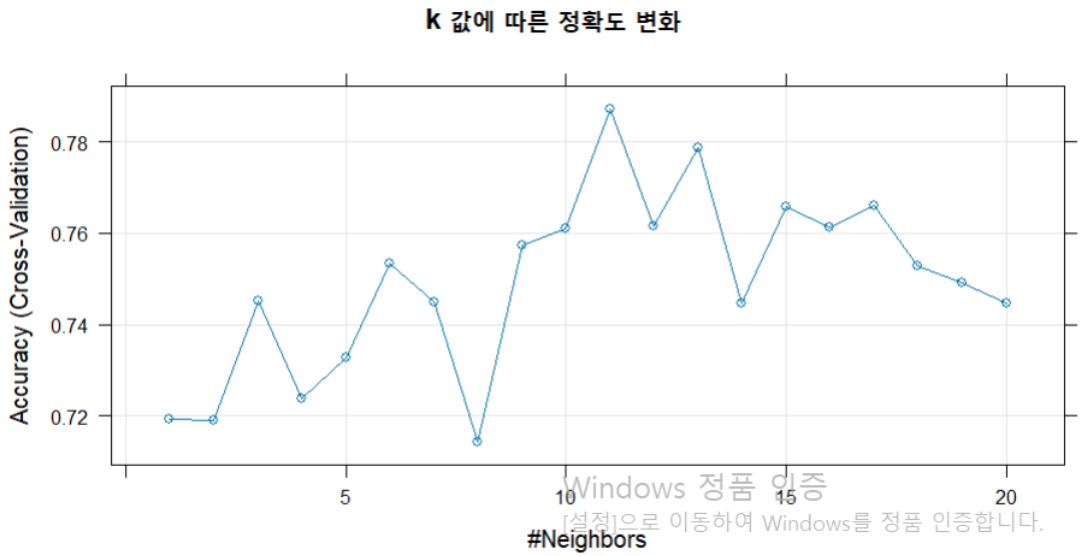
```r
# Q4: 최적의 k 찾기
if (!requireNamespace("caret", quietly = TRUE)) {
  install.packages("caret")
}
library(caret)

# 데이터 표준화
preproc <- preProcess(train_x, method = c("center", "scale"))
train_x_norm <- predict(preproc, train_x)
valid_x_norm <- predict(preproc, valid_x)

# 최적의 k 값 찾기
control <- trainControl(method = "cv", number = 10)  # 10-폴드 교차 검증
model <- train(
  x = train_x_norm,
  y = train_y,
  method = "knn",
  trControl = control,
  tuneGrid = expand.grid(k = 1:20)  # k 값 탐색 범위
)

# 최적의 k 값 출력
optimal_k <- model$bestTune$k
optimal_accuracy <- max(model$results$Accuracy)
cat("Q4: 최적의 k:", optimal_k, "\n최적의 정확도:", optimal_accuracy, "\n")

# Step 2: 결과 시각화
plot(model, main = "k 값에 따른 정확도 변화")
```



Windows 정품 인증
[설정]으로 이동하여 Windows를 정품 인증합니다.

① caret 패키지를 활용하여 10-폴드 교차 검증으로 최적의 k 값을 탐색

②
```
Q4: 최적의 k: 11
최적의 정확도: 0.7871377
```

Q5. Test 데이터에 대해 kNN 분류

```r
# Q5: Test 데이터에 대한 kNN 분류
test_pred <- knn(
  train = as.matrix(train_x),  # 학습 세트의 독립 변수
  test = as.matrix(test_x),    # 테스트 세트의 독립 변수
  cl = train_y,                # 학습 세트의 종속 변수 (레이블)
  k = optimal_k                # 최적의 k 값 사용
)

# 테스트 세트 정확도 계산
accuracy_q5 <- sum(test_pred == test_y) / length(test_y)
cat("Q5: 최적의 k=", optimal_k, "에서 테스트 세트의 정확도:", accuracy_q5, "\n")

# 예측 값과 실제 값 비교
comparison_q5 <- data.frame(
  Actual = test_y,
  Predicted = test_pred
)
print(comparison_q5)  # 실제 값과 예측값 확인

# Confusion Matrix (테스트 세트)
confusion_q5 <- confusionMatrix(factor(test_pred), factor(test_y))
print(confusion_q5)
```

① 최적의 k 값을 사용하여 테스트 세트를 예측한 결과를 도출

최적의 k: 11

② **결과:** 테스트 세트의 정확도: **84.81%**

```
> # Q5: Test 데이터에 대한 kNN 분류
> test_pred <- knn(
+   train = as.matrix(train_x),  # 학습 세트의 독립 변수
+   test = as.matrix(test_x),    # 테스트 세트의 독립 변수
+   cl = train_y,                # 학습 세트의 종속 변수 (레이블)
+   k = optimal_k                # 최적의 k 값 사용
+ )
>
> # 테스트 세트 정확도 계산
> accuracy_q5 <- sum(test_pred == test_y) / length(test_y)
> cat("Q5: 최적의 k=", optimal_k, "에서 테스트 세트의 정확도:", accuracy_q5, "\n")
Q5: 최적의 k= 11 에서 테스트 세트의 정확도: 0.8481013
```

Q5. Test 데이터에 대해 kNN 분류

```
> # Confusion Matrix (테스트 세트)
> confusion_q5 <- confusionMatrix(factor(test_pred), factor(test_y))
> print(confusion_q5)
Confusion Matrix and Statistics

          Reference
Prediction neg pos
       neg  50   6
       pos   6  17

               Accuracy : 0.8481
                 95% CI : (0.7497, 0.919)
    No Information Rate : 0.7089
    P-Value [Acc > NIR] : 0.003098

                  Kappa : 0.632

 Mcnemar's Test P-Value : 1.000000

            Sensitivity : 0.8929
            Specificity : 0.7391
         Pos Pred Value : 0.8929
         Neg Pred Value : 0.7391
             Prevalence : 0.7089
         Detection Rate : 0.6329
   Detection Prevalence : 0.7089
      Balanced Accuracy : 0.8160

       'Positive' Class : neg
```

	Actual	Predicted
1	neg	neg
2	neg	neg
3	pos	pos
4	neg	neg
5	pos	neg
6	neg	neg
7	neg	neg
8	neg	neg
9	neg	neg
10	neg	neg
11	neg	neg
12	neg	neg
13	neg	neg
14	neg	neg
15	neg	neg
16	neg	neg
17	neg	neg
18	neg	pos
19	neg	pos
20	pos	pos
21	pos	pos
22	neg	neg
23	pos	pos
24	pos	pos
25	pos	pos
26	neg	neg
27	pos	neg
28	neg	pos
29	pos	pos
30	neg	neg
31	neg	neg
32	neg	neg
33	neg	neg
34	neg	neg
35	neg	neg
36	neg	neg
37	pos	pos
38	neg	neg
39	neg	neg

① confusion matrix 결과:

민감도(Sensitivity): 89.29% 특이도(Specificity): 73.91%

-> neg 클래스를 더 잘 예측했으며, pos 클래스의 예측은

약간 낮은 특이도를 보여줍니다.

* 민감도: neg 클래스(비당뇨병)의 관측값 중 올바르게 예측한 비율

-> 모델이 비당뇨병을 매우 잘 예측하고 있음을 보여

* 특이도: pos 클래스(당뇨병)의 관측값 중 올바르게 예측한 비율

-> 상대적으로 낮은 특이도는 모델이 당뇨병을 예측할 때

약간의 어려움을 겪고 있음을 의미함.

-> 특이도가 낮은 것은 당뇨병 환자를 "놓칠 가능성"을 내포하므로,

의료 분야와 같은 민감한 상황에서는 위험할 수 있음.